Databases I

Fundamentals of Database Systems





Objectives

- **The course aims to:**
- Understand the basic concepts of database systems, their purpose, and the architecture of a database management system (DBMS).
- Learn about different data models, focusing on the relational model, and understand its advantages and limitations.
- Grasp the principles of data protection, including data security and integrity, ensuring data is accurate and consistent.
- Use SQL commands for database construction (DDL) and data manipulation (DML).
- Apply normalization techniques to design efficient and well structured databases.

Chapter 1

Introduction



Outline of lecture

- **Databases & Databases Management Systems**
- **Database-System Applications**
- **Describing & Storing Data in a DBMS**
- **STRUCTURE OF A DBMS**
- **Database Access from Application Programs**

BASIC CONCEPTS AND DEFINITIONS

- Data: facts concerning objects and events that could be recorded and stored on computer media. (e.g. facts, text, graphics, images, sound, video segments)
- Information: Data processed to be useful in decision making

Data vs Information

Baker, Kenneth D.	324917628
Doyle, Joan E.	476193248
Finkle, Clive R.	548429344
Lewis, John C.	551742186
McFerran, Debra R.	409723145
Sisneros, Michael	392416582

Data in Context

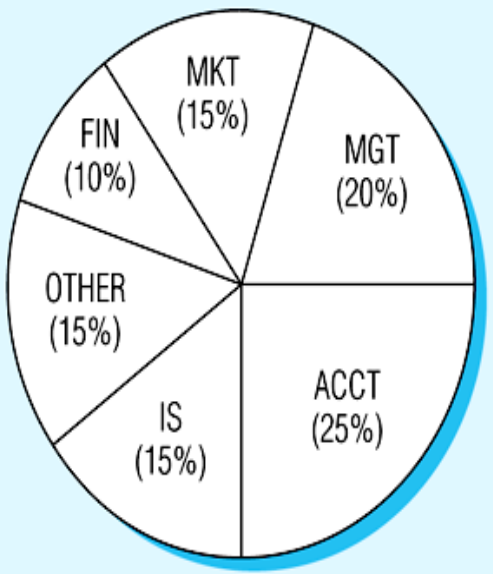
Class Roster

Course: MGT 500 Business Policy Semester: Spring 200X

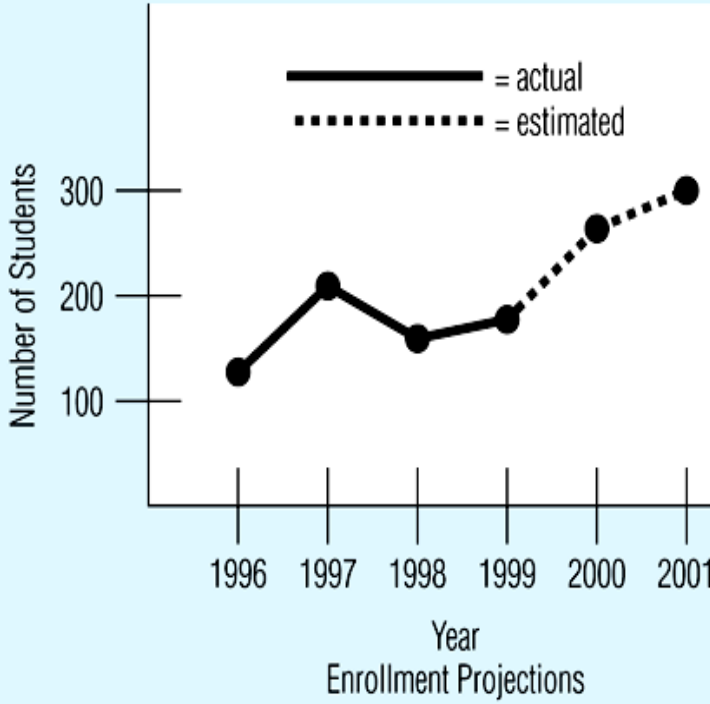
Section: 2

Name	ID	Major	GPA
Baker, Kenneth D.	324917628	MGT	2.9
Doyle, Joan E.	476193248	MKT	3.4
Finkle, Clive R.	548429344	PRM	2.8
Lewis, John C.	551742186	MGT	3.7
McFerran, Debra R.	409723145	IS	2.9
Sisneros, Michael	392416582	ACCT	3.3

Summarized data



Percent Enrollment by Major (199X)



Enrollment Projections

Metadata

Metadata: Data that describes data

Descriptions of the properties or characteristics of the data, including data types, field sizes, allowable values, and documentation

Table 1-1 Example Metadata for Class Roster

Data Item		*Value*			
Name	**Type**	**Length**	**Min**	**Max**	**Description**
Course	Alphanumeric	30			Course ID and name
Section	Integer	1	1	9	Section number
Semester	Alphanumeric	10			Semester and year
Name	Alphanumeric	30			Student name
ID	Integer	9			Student ID (SSN)
Major	Alphanumeric	4			Student major
GPA	Decimal	3	0.0	4.0	Student grade point average

BASIC CONCEPTS AND DEFINITIONS …

- **Database:** An organized collection of logically related data
  - This data is a description of the activities within an organization or a group of interconnected organizations

database *is a collection of entities and the relationships between them*

Example (University Database): If we take the university database as an example, we find that the **entities** are the *students*, *professors*, *courses*, and study locations (*classrooms*), and the **relationships** are the relationship of *students to courses*, the relationships of *professors to courses*, and the relationship of *courses to classes*.

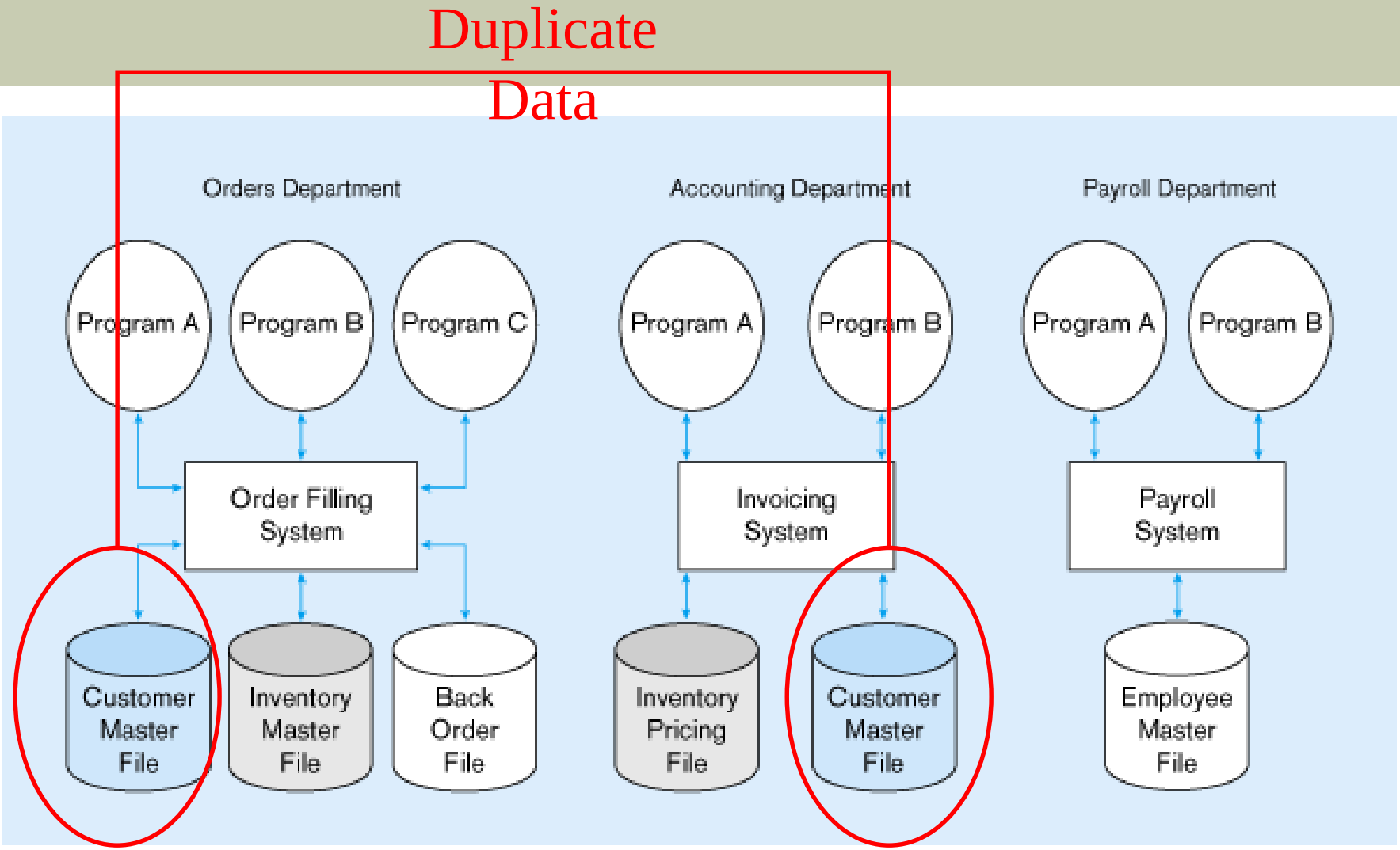
Types of Databases and Database Applications

- Traditional Applications:
  - Numeric and Textual Databases
- More Recent Applications:
  - Multimedia Databases
  - Geographic Information Systems (GIS)
  - Data Warehouses
  - Real-time and Active Databases
  - Many other applications
- this course focuses on traditional applications

TRADITIONAL FILE PROCESSING SYSTEMS

- computers had to store, manipulate, and retrieve large files of data
- Computer file processing systems were developed for this purpose
- As business applications became more complex, it became evident that traditional file processing systems had a number of shortcomings and limitations

Three file processing systems at Pine Valley Furniture



Disadvantages of File Processing

Program-Data Dependence

- Each application programmer must maintain their own data.
- Each application program needs to include code for the metadata of each file.
- Each application program must have its own processing routines for reading, inserting, updating and deleting data.
- Lack of coordination and central control
- Non-standard file formats.

Limited Data Sharing

- No centralized control of data

Disadvantages of File Processing(cont.)

Data Redundancy (Duplication of data)

- Waste of space to have duplicate data
- Causes more maintenance headaches
- The biggest Problem:
  - **When data changes in one file, could cause inconsistencies**
  - Compromises ***data integrity***

Lengthy Development Times

- Programmers must design their own file formats

Excessive Program Maintenance

- 80% of of information systems budget

Operating system

Why not use **Operating System (OS)**, which is one of the file management systems, to manage database files?

Operating System (OS) works as follows:

- **First: OS** transfers the entire data file to the main memory (RAM) if users want to deal with any part of the data within a file
- **Second: OS** secures data through a *password mechanism*. This strategy does not provide sufficient flexibility when implementing the aforementioned data security
- **Third:** In **OS**, special programs (scripts) must be written to answer each question a user may want to ask about the data.

SOLUTION:
The DATABASE Approach

Central repository of shared data

Data is managed by a controlling agent

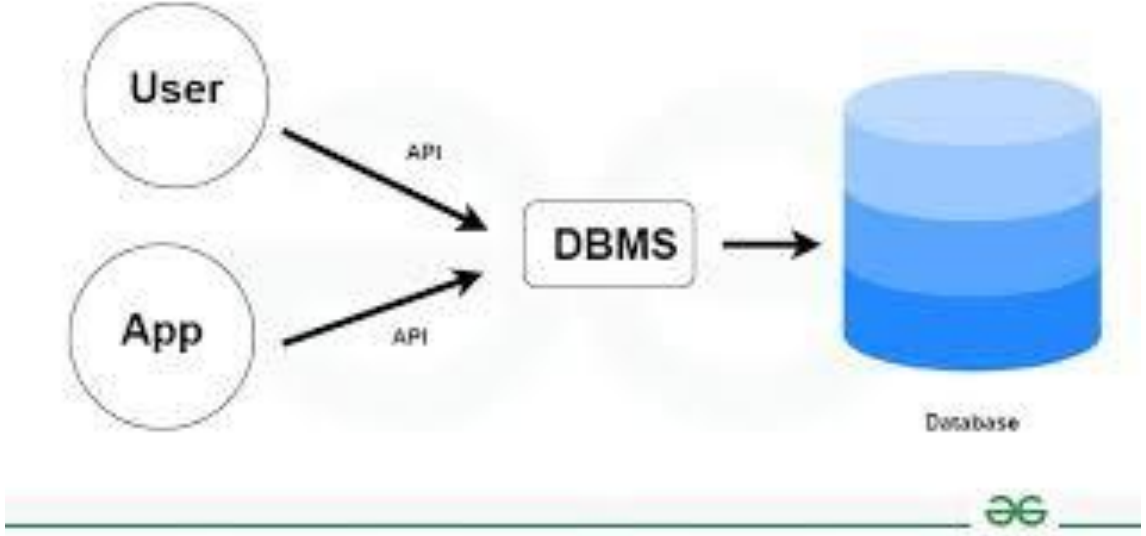
Stored in a standardized, convenient form

Requires a Database Management System (DBMS)

Database Management System (DBMS):

❑ Database Management System (DBMS):

- A software package/ system to facilitate the creation and maintenance of a computerized database.



OS Vs DBMS

- **OS** consider a **file** as *a string of characters*, whereas with **DBMSs**, the **file** has become a *collection of pages, each page being a collection of records*.

- **The second defect facing OS,** the creation of a program to answer every question, was overcome in **DBMSs** by developing a language called (**SQL**).

Advantages of Database Approach

- Program-Data Independence
- Minimal Data Redundancy
- Improved Data Sharing
  - Different users get different views of the data
- Enforcement of Standards
  - All data access is done in the same way
- Improved Data Quality
  - Constraints, data validation rules
- Better Data Accessibility/ Responsiveness
  - Use of standard data query language (SQL)
- Security, Backup/Recovery, Concurrency
  - Disaster recovery is easier

*Challenges of **DBMS***

- The database of any organization often consists of a *huge amount of data* that sometimes reaches **hundreds of GBs or terabytes**.
- *large number of users* can access the data at the same time
- ***DBMS must be***

➢ **Fast:** very fast when performing any operation by users.

➢ **Integrable:** able to maintain the integrity and compatibility of the data after performing any operation on the data.

➢ **Secure:** There is also an urgent need for ensuring data security and privacy, as the system **must provide data to groups of users**, each according to their **permissions** within the organization. Some users have permission to access all data, while others have permission to access only a small portion of it.

Database-System Applications

Databases are widely used. Here are some representative applications:

- **Enterprise Information**
  - **Sales, accounting, human resources, Manufacturing,…**
- **Banking and Finance**
  - **Banking. Credit card transactions, Finance,…**
- **Universities**
- **Airlines**

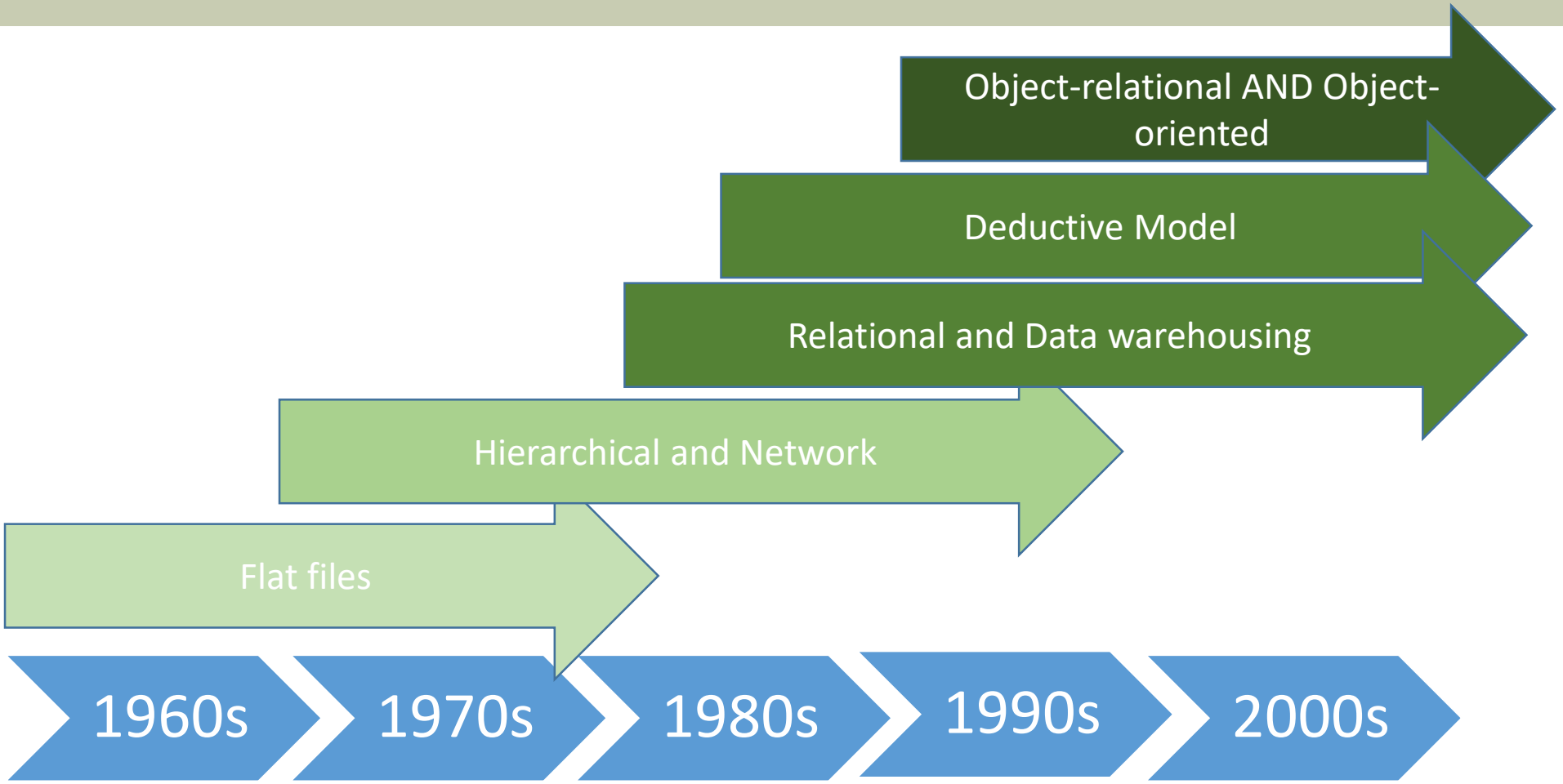
Describing & Storing Data in a DBMS

- A **data model** is *a collection of high-level data description constructs that hide many low-level storage details*

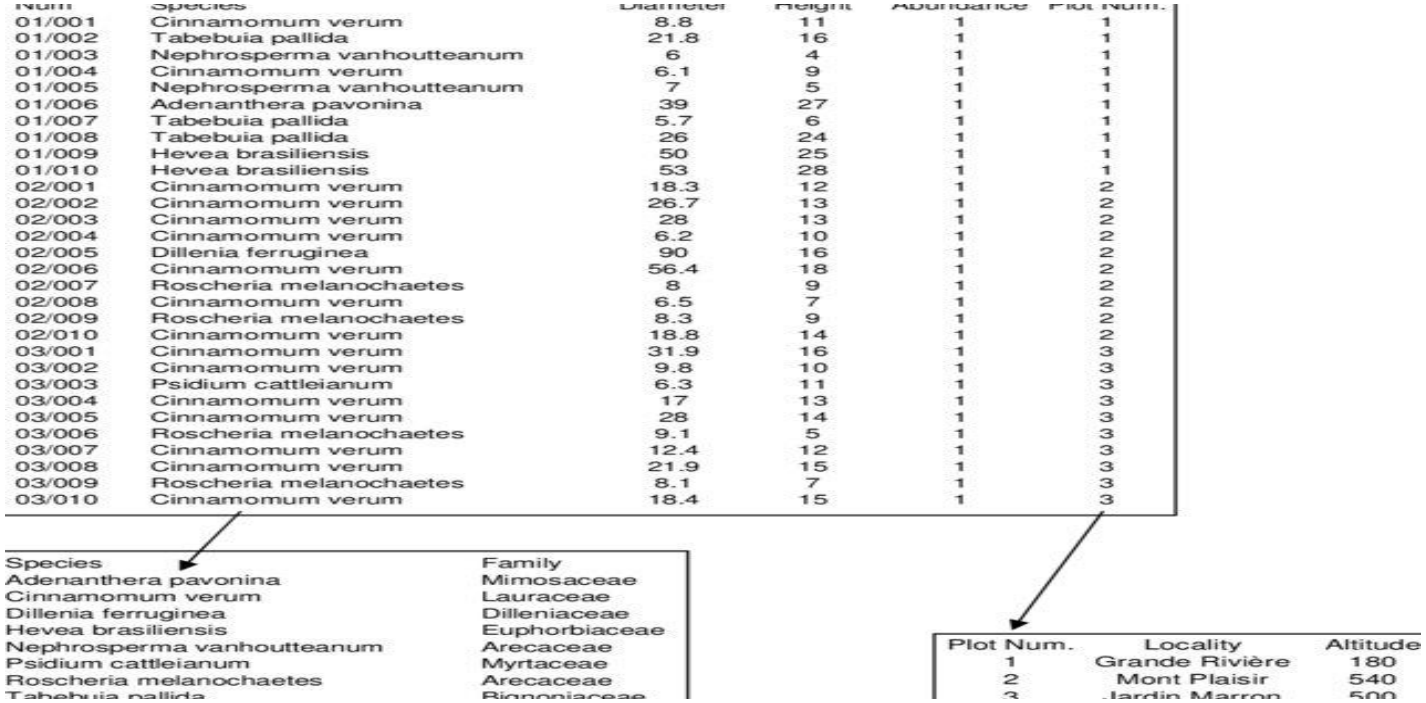
- A **semantic data model** is *a more abstract, high-level data model that helps a database designer to come up with a good initial description of the data in an enterprise*
  - A type of data model that focuses on **capturing the meaning (semantics)** of data, not just the structure.
- **Entity-Relationship (ER) model** *is a widely used semantic data model*
- **Relational data model is a** data model where data is organized into **tables (relations)** with rows (tuples) and columns (attributes).

- Most DBMS today, including IBM's DB2, Oracle, etc., are based on the **relational data model**

Evolution of DB Systems



Flat database



Num	Species	Diameter	Height	Abundance	Plot Num.
01/001	Cinnamomum verum	8.8	11	1	1
01/002	Tabebuia pallida	21.8	16	1	1
01/003	Nephrosperma vanhoutteanum	6	4	1	1
01/004	Cinnamomum verum	6.1	9	1	1
01/005	Nephrosperma vanhoutteanum	7	5	1	1
01/006	Adenanthera pavonina	39	27	1	1
01/007	Tabebuia pallida	5.7	6	1	1
01/008	Tabebuia pallida	26	24	1	1
01/009	Hevea brasiliensis	50	25	1	1
01/010	Hevea brasiliensis	53	28	1	1
02/001	Cinnamomum verum	18.3	12	1	2
02/002	Cinnamomum verum	26.7	13	1	2
02/003	Cinnamomum verum	28	13	1	2
02/004	Cinnamomum verum	6.2	10	1	2
02/005	Dillenia ferruginea	90	16	1	2
02/006	Cinnamomum verum	56.4	18	1	2
02/007	Roscheria melanochaetes	8	9	1	2
02/008	Cinnamomum verum	6.5	7	1	2
02/009	Roscheria melanochaetes	8.3	9	1	2
02/010	Cinnamomum verum	18.8	14	1	2
03/001	Cinnamomum verum	31.9	16	1	3
03/002	Cinnamomum verum	9.8	10	1	3
03/003	Psidium cattleianum	6.3	11	1	3
03/004	Cinnamomum verum	17	13	1	3
03/005	Cinnamomum verum	28	14	1	3
03/006	Roscheria melanochaetes	9.1	5	1	3
03/007	Cinnamomum verum	12.4	12	1	3
03/008	Cinnamomum verum	21.9	15	1	3
03/009	Roscheria melanochaetes	8.1	7	1	3
03/010	Cinnamomum verum	18.4	15	1	3

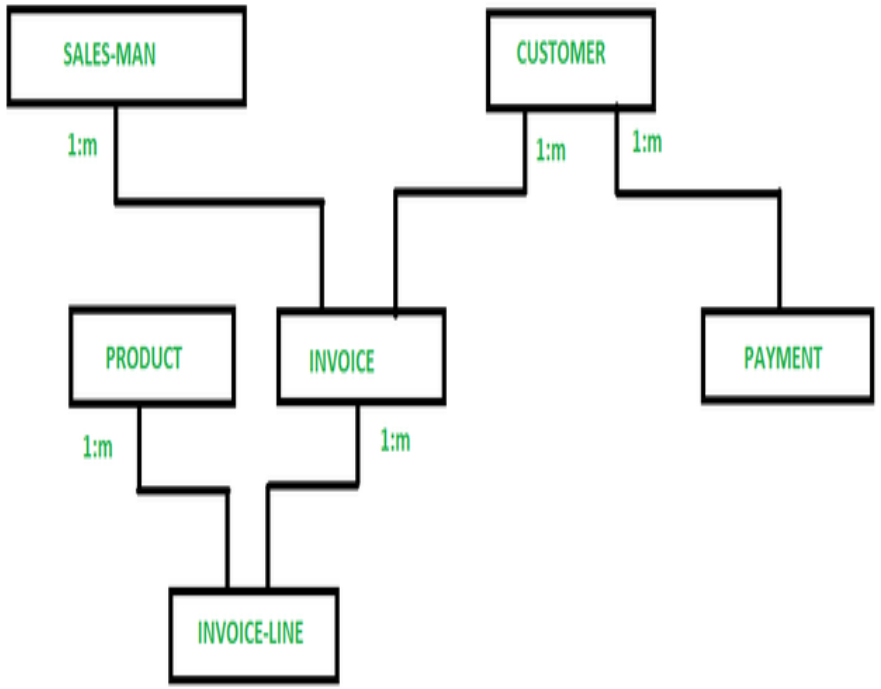
Species	Family
Adenanthera pavonina	Mimosaceae
Cinnamomum verum	Lauraceae
Dillenia ferruginea	Dilleniaceae
Hevea brasiliensis	Euphorbiaceae
Nephrosperma vanhoutteanum	Arecaceae
Psidium cattleianum	Myrtaceae
Roscheria melanochaetes	Arecaceae
Tabebuia pallida	Bignoniaceae

Plot Num.	Locality	Altitude
1	Grande Rivière	180
2	Mont Plaisir	540
3	Jardin Marron	500

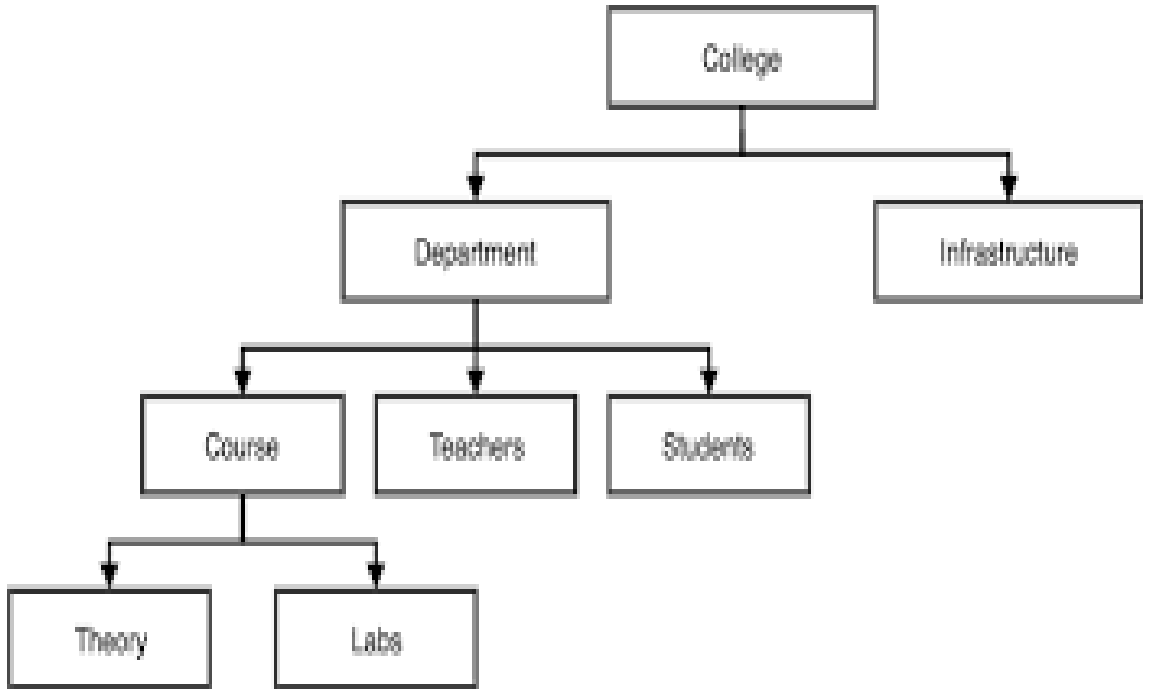
Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Network Model &**Hierarchical Model**

Network Model



Hierarchical Dataase



Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Relational & Object-oriented database model

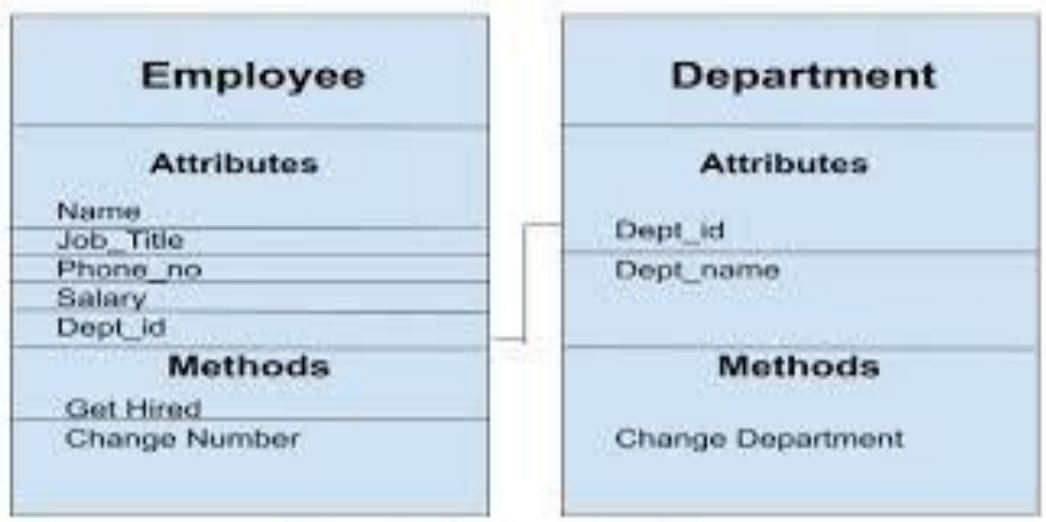
Relational database model

student_id	name	age
1	Akon	17
2	Bkon	18
3	Ckon	17
4	Dkon	18

subject_id	name	teacher
1	Java	Mr. J
2	C++	Miss C
3	C#	Mr. C Hash
4	Php	Mr. P H P

student_id	subject_id	marks
1	1	98
1	2	78
2	1	76
3	2	88

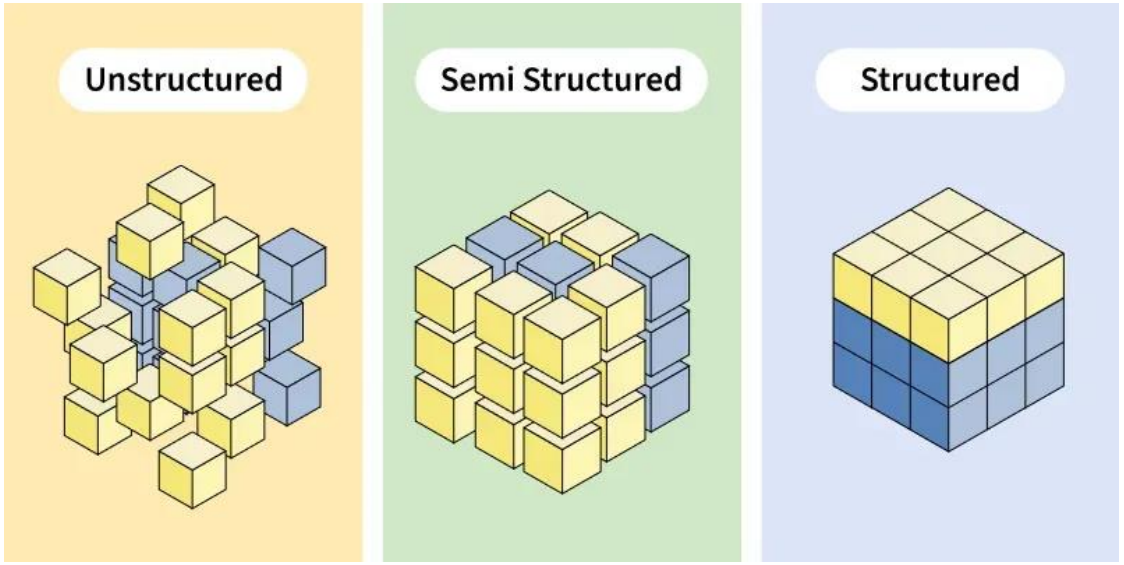
Object-oriented database model

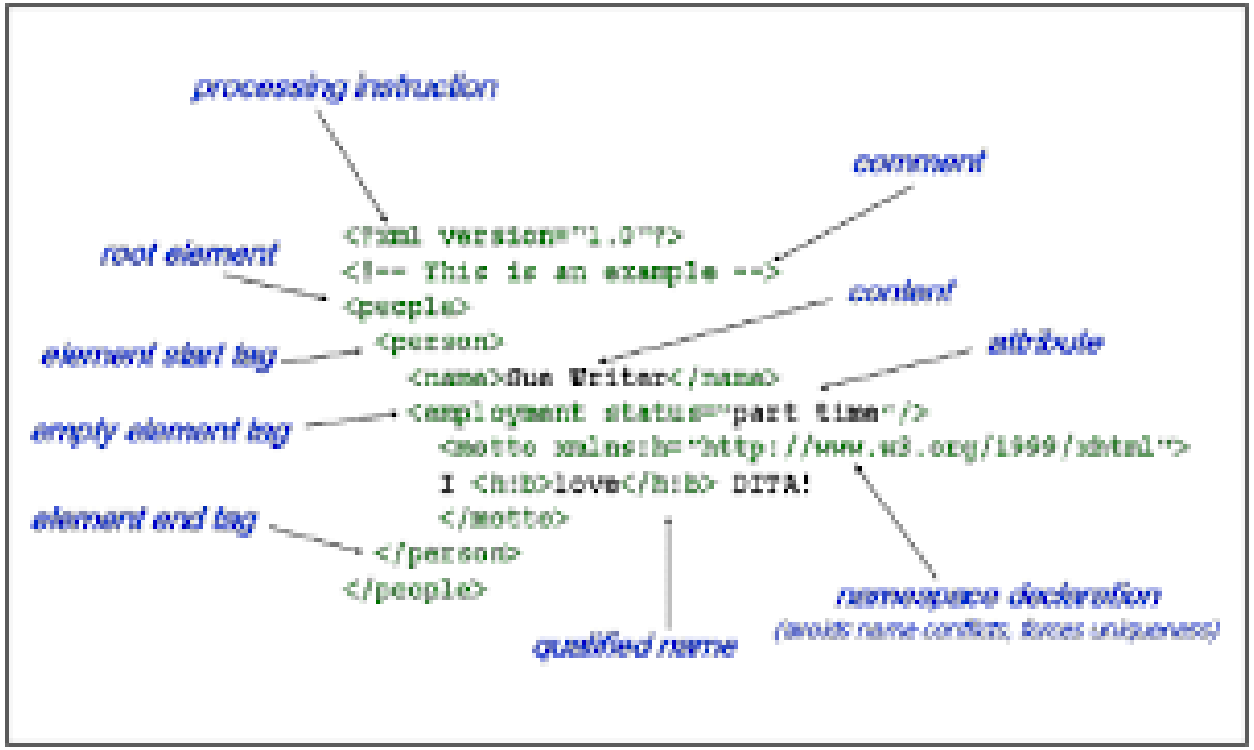


Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

semi-structured data model(early 2000s)

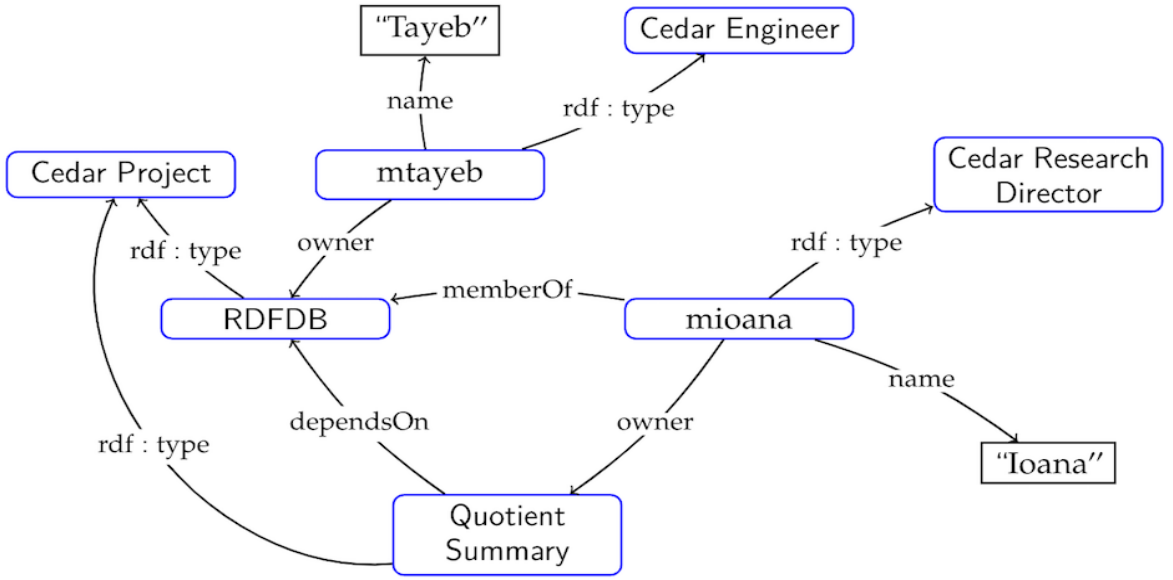
- permits the specification of data where individual data items of the same type may have different sets of attributes
- The **Extensible Markup Language (XML)** is widely used to represent semi-structured data.





RDF data model

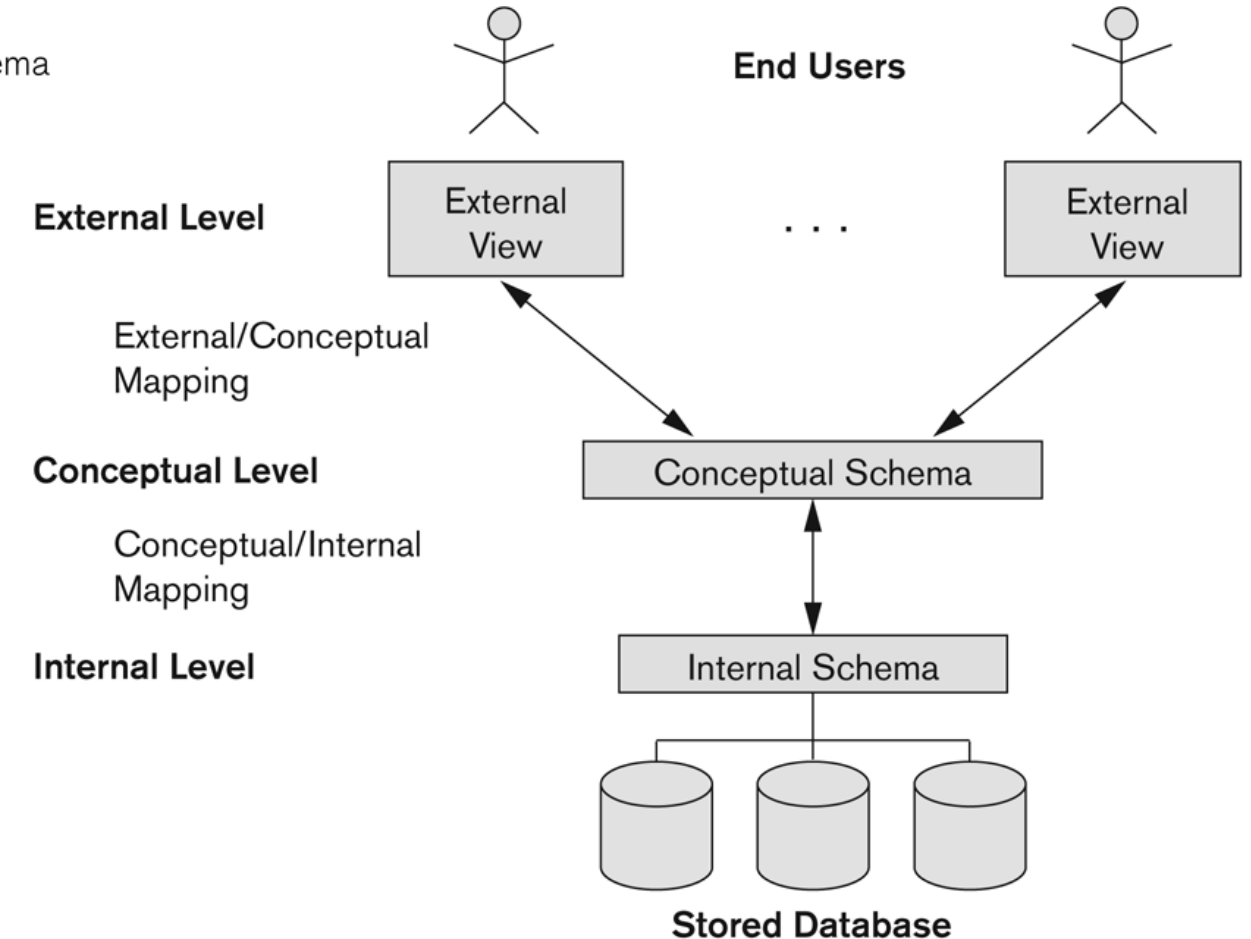
- Recently, the **RDF data model** provides a standard, machine readable way to publish and share data on the web, forming the basis of the **Semantic Web** and **knowledge graphs**



Level of Data Description in a DBMSs:

Data independence is achieved in DBMSs by *describing data at three Levels of abstractions* which are *conceptual*, *physical*, and *external*

Figure 2.2
The three-schema architecture.



Three-Schema Architecture

External Level (User Views)

- Closest to the **end users**.
- Shows **how each user sees the data**.
- There can be **many external schemas** (one per user group).
- Example:
  - A student sees only their grades.
  - A lecturer sees all students' grades in their course.

The three-schema architecture…

Conceptual Level (Community View)

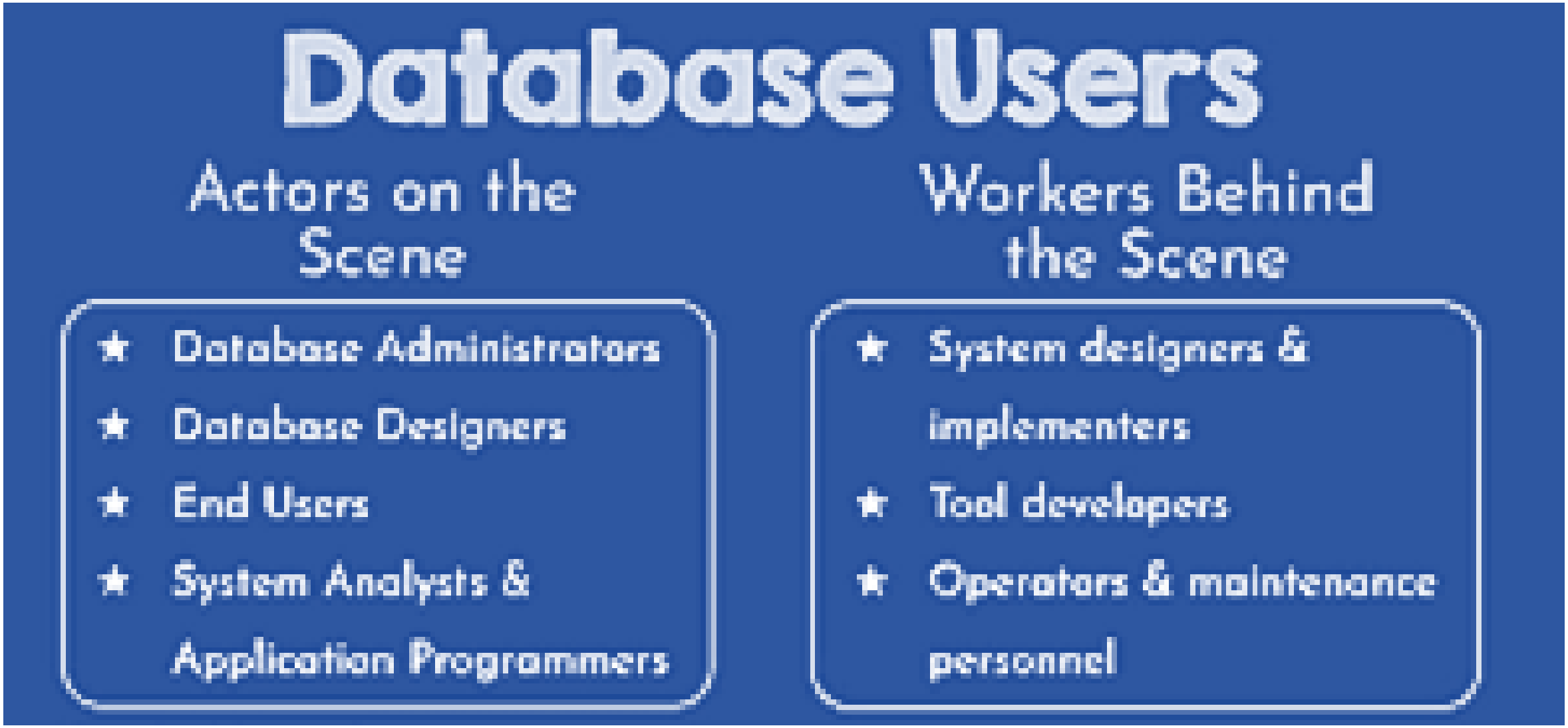
- A **unified, global view** of the entire database.
- Describes **all entities, attributes, and relationships** without worrying about physical storage.
- Represents the **semantic model** of the organization's data.
- Example:
  - Students, Courses, Instructors, Enrollments, and their relationships.
- **Key Point**: Hides **physical storage details**, but shows the **full logical structure**.

The three-schema architecture…

Internal Level (Physical Storage)

- Lowest level; closest to how data is actually stored in the database.
- Defines **data structures, indexes, file paths, storage blocks**.
- Example:
  - Tables, indexes, B-trees, hash structures.
- **Key Point**: Handles **performance and efficiency**, hidden from users.

Database Users



Database Users

- DBMS system designers and implementers :*Build DBMSs software*
- End Users: *Who wish to store and use data in DBMS*
- Database Application programmers: *Develop packages that facilitate data access for end users*
- Database Administrator (DBA): *Responsible for many tasks*
  - *Design databases(designing the data model)*
- Security & Authorization: *Ensures that unauthorized data access is not permitted.*
- Database Tuning: *Modifies the conceptual and physical schemas as user's needs evolve with time.*
  - Data Availability and Recovery from Failure

Database Languages: Talking with DBMSs

- SQL, stands for Structured Query Language, the standard language for creating, manipulating and querying data in DBMS
- Data-Definition Language (DDL)
  - supports creation, deletion, and modification of conceptual database schema;
- The output of the **DDL** is placed in the **data dictionary**, *which contains* ***metadata***—that is, *data about data*
- The **data dictionary** is considered to be *a special type of files that can only be accessed and updated by the database system itself (not a regular user)*
- *The database system consults the data dictionary before reading or modifying actual data.*

Database Languages: Talking with DBMSs

- Data-Manipulation Language (DML)
  - DML enables users to access or manipulate data as organized by the appropriate data model.
  - The types of access are:

1. Retrieval of information stored in the database. (query)
2. Insertion of new information into the database.
3. Deletion of information from the database.
4. Modification of information stored in the database. (update)

Integrity Constraints

- Integrity Constraints (ICs) in general are conditions, specified on the database schema, that the data stored in the database must satisfy
- **ICs** are *specified when creating the database while* are *enforced when a database application is run*

1. **Domain Constraints**: A domain of possible values must be associated with every attribute of database entities (for example, integer types, character types, date/time types)
   - Declaring an attribute to be of a particular domain acts as a constraint on the values that it can take.
2. **Referential Integrity**: There are cases where we wish to ensure that a value that appears in one entity for a given set of attributes also appears in a certain set of attributes in another entity ((**foreign keys must reference existing rows**).
3. **Entity Integrity Constraint:** Ensures every entity has a **unique and non-null identifier** (primary key).

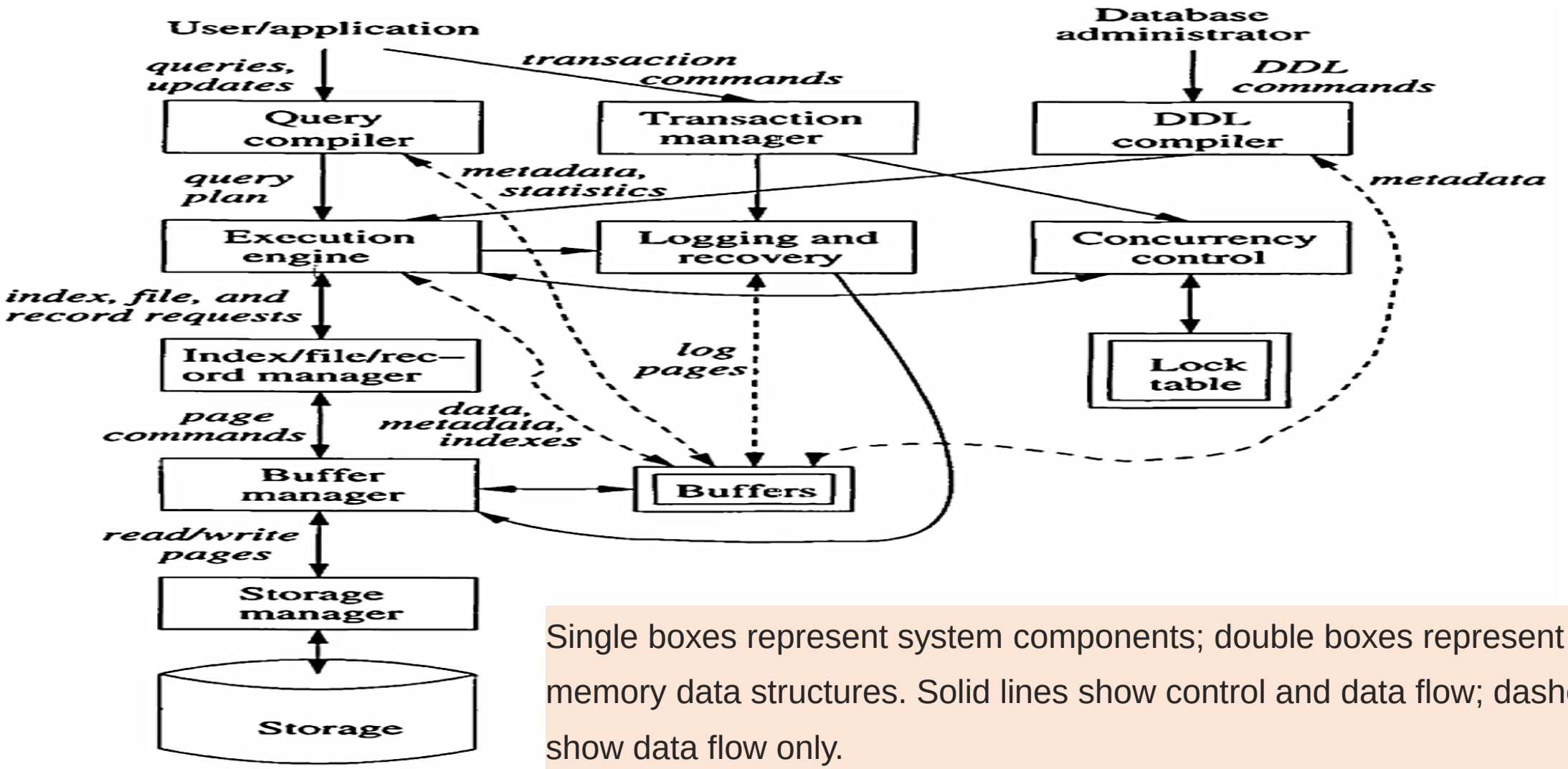
Integrity Constraints…

- Assertions: An assertion is any condition that the database must always satisfy. Domain constraints and referential-integrity constraints are special forms of assertions. When an assertion is created, the system tests it for validity.
- Authorization: We may want to differentiate among the users as far as the type of access they are permitted on various data values in the database
  - read authorization
  - insert authorization
  - update authorization
  - delete authorization

Integrity Constraints…

- Integrity constraints are specified and enforced at different times:
  - When the **DBA** defines a database schema, he/she specifies ICs that must hold on any instance of this database.
  - When a database application is run, the DBMS checks for violations and disallows changes to the data that violate the specified ICs

STRUCTURE OF A DBMS



Single boxes represent system components; double boxes represent in-memory data structures. Solid lines show control and data flow; dashed lines show data flow only.

Fig. 1.2 : Database management system components

Two Command Sources

DDL Commands

Issued by database administrators. Responsible for database structure and schema. Require special authority due to profound effects on the database.

Example: DBA of a university registrar's database might decide that allowable grades (A, B, C, D, F) for courses.

Query Commands

Issued by end users and application programs. Use data manipulation language (DML). Do not affect schema but may modify or extract database content.



DDL Processing

1

DDL Command

Schema-altering instruction from DBA

2

DDL Processor

Parses command and passed to the execution engine

3

Execution Engine

Processes through managers

4

Metadata Update

Schema information altered

Query processing commands

- A user or an application program initiates some action, using the data-manipulation language (DML)

- DML command does not affect the schema of the database, but may affect the content of the database (if the action is a modification command) or will extract data from the database (if the action is a query)

DML commands are handled by two separate subsystems, as follows.

Query Answering

Transaction Processing

Query Processing: Query Answering

01

Query Compiler

Parses and optimizes query, producing a query plan—a sequence of actions the DBMS performs to answer the query.

02

Execution Engine

Receives query plan and issues requests for data pieces to the File Access Manager.
that knows about data files such as format and size of data elements and indices, which helps find elements of data files quickly

03

Buffer Manager

Manages page-sized memory regions(**buffers**) where disk blocks transfer. Communicates with storage manager.

04

Disk Controller

Retrieves data from disk to buffers in main memory for processing.

Five Types of Information

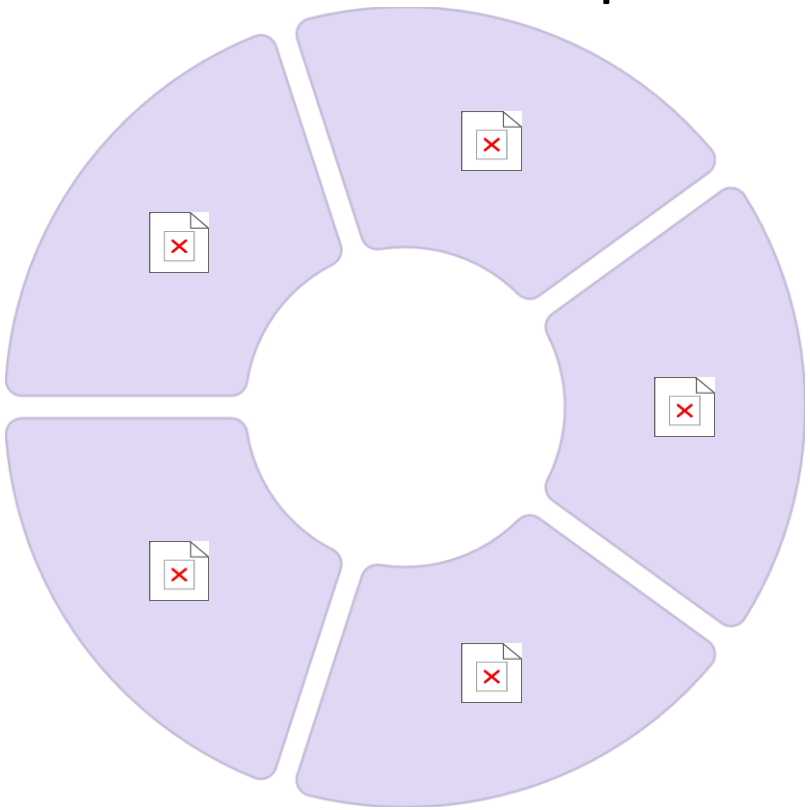
- The kinds of information that various components may need include:

Data

Database contents

Indexes

Efficient data access structures



Metadata

Schema and constraints

Log Data

Recent changes for durability

Statistics

Data properties and sizes

Query Processing: Transaction Processing

- **A Transaction** is a group of one or more database operations which is a unit of work that must be executed atomically and in apparent isolation from other transactions.
- The transaction manager therefore accepts transaction commands from an application, which tell the transaction manager when transactions begin and end, as well as information about the expectations of the application

The transaction processor performs the following tasks:

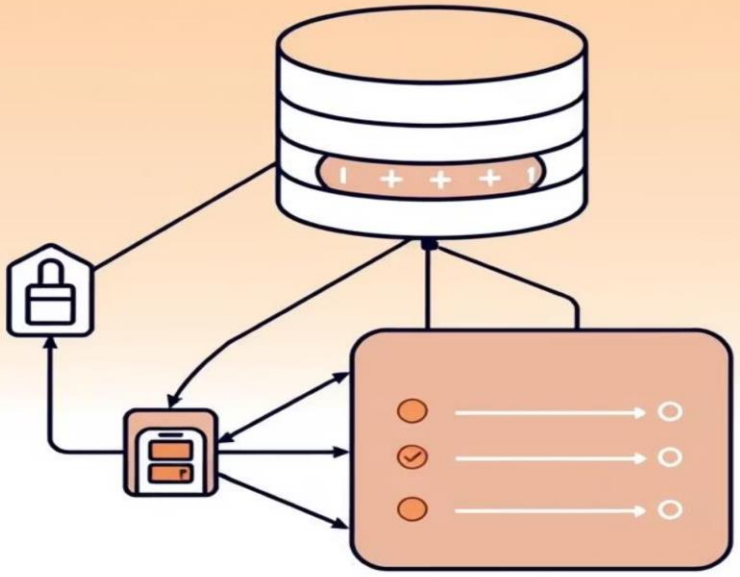
Logging

Concurrency control

Deadlock resolution

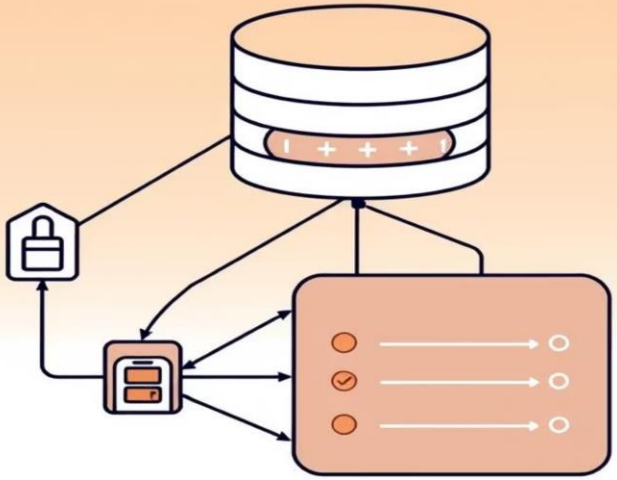
Logging

- In order to assure , every change in durability the database is logged separately on disk
- The log manager follows one of several policies designed to assure that no matter when a system failure or “crash” occurs
- a recovery manager will be able to examine the log of changes and restore the database to some consistent state.



Concurrency Control

- responsible for ensuring that transactions are executed **concurrently** (at the same time) **without interfering** with each other and that **data integrity and consistency** are maintained.
- A typical scheduler does its work by maintaining locks on certain pieces of the database
- A **lock** is a mechanism the **Concurrency-Control Manager** uses to **control access** to data in a multi-user database system.
- Locks are generally stored in a main-memory lock table,
- When a transaction *locks* a data item: It is **reserving** that item for its own use. Other transactions must **wait** until the lock is **released**.



Concurrency Control…

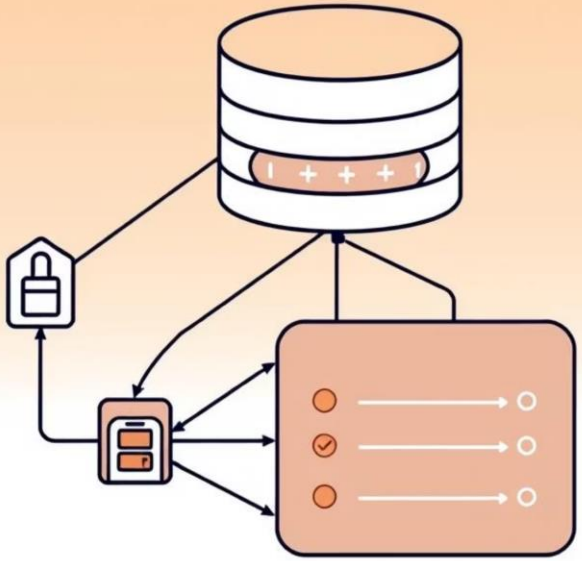
- **E.g. Scenario: Bank Database Example**
- Two transactions are running at the same time:
- **T1:** Transfers money from **Account A → Account B**
- **T2:** Transfers money from **Account B → Account A**

T1 locks Account A”, it means that the **transaction T1** has **exclusive control** over the data item **Account A** inside the database — and **no other transaction** is allowed to modify (or sometimes even read) that data until T1 finishes.

Deadlock Resolution

- When transactions compete for locked resources and cannot proceed, transaction manager intervenes by rolling back one or more transactions.
- uses to **handle and recover** from a **deadlock** — a situation where two or more transactions are **waiting for each other's locked resources indefinitely**, so none of them can proceed.

➢ Deadlock Detection and Recovery



Deadlock Resolution…

- **Recovery Actions:**
- **Transaction Rollback (Abort):**
  - The system **chooses a victim transaction** (the one to abort).
  - Rolls it back to free its resources.
  - The victim can be restarted later.
- **Victim selection criteria**:
  - Least progress made,
  - Smallest cost to rollback,
  - Fewest resources used.
- **Preemption:**
  - Temporarily **take resources** from some transactions and **give them to others** to break the cycle.

Deadlock Resolution…

- 1 waits for T2 to release **B**
- T2 waits for T1 to release **A**
- ⟶ **Neither can continue → Deadlock!**

Resolution (Breaking the Deadlock)

- The DBMS chooses a **victim transaction** to abort.
  Let's say it aborts **T2** (because it has done less work).
- Steps:
- Abort **T2** and release all its locks (A and B).
- **T1** can now acquire lock **B**, complete its transfer, and commit.
- Restart **T2** later if needed.

Query Processing: Transaction Processing

ACID Properties of a Transaction

e.g.Suppose that a transaction is a funds transfer, in which one department account (say A) is debited and another department account (say B) is credited

Atomicity

Either all operations complete or none occur. Debit account A and credit account B must both succeed or both fail.

Consistency

Database remains valid. Sum of balances in A and B is preserved throughout the transaction.

Isolation

Transactions run *independently*, as if each runs alone — no interference.

Durability

Once a transaction *commits*, its changes are *permanent*, even after a crash..

New account balances survive crashes.

Database Access from Application Programs

- SQL is a declarative language and not powerful as procedural ones.
- SQL also does not support actions such as input from users, output to displays, or communication over the network
- Such computations and actions must be written in a host language(as C, C++, or Java, with embedded SQL queries that access the data in the database)

Database Access from Application Programs…

- To access the database, DML statements need to be executed from the host language. There are two ways to do this:

1. **Procedure Call Approach**: By providing an application program interface (set of procedures) that can be used to send DML and DDL statements to the database and retrieve the results.
   - The Open Database Connectivity (ODBC) standard for use with the C/C++ language is a commonly used application program interface standard.
   - The Java Database Connectivity(JDBC) standard provides corresponding features to the Java language

Database Access from Application Programs…

2. **Embedded Approach**: e.g embedded SQL (for C, C++, etc.), SQLJ (for Java)

- By extending the host language syntax to embed DML calls within the host language program. Usually, a special character prefaces DML calls, and a preprocessor, called the DML pre-compiler, converts the DML statements to normal procedure calls in the host language

Database Programming Language Approach: e.g. ORACLE has PL/SQL, a programming language based on SQL; language incorporates SQL and its data types as integral components

Database Access from Application Programs…

pseudo-code for two approaches

```
conn = db_connect("SchoolDB");
result = db_execute(conn, "SELECT * FROM Student");
while (row = db_fetch(result)) {
   print(row["Name"]);
}
```

procedure calls (API functions) to talk to the DB

```
EXEC SQL CONNECT TO SchoolDB USER 'admin' IDENTIFIED BY '123';
EXEC SQL SELECT Name
        INTO :student_name
        FROM Student
        WHERE Student_ID = :id;
```

SQL is **directly embedded** in the host language

client machines & server machines

- Most users of a DBMS today are not present at the site of the system, but connect to it through a network (Client–Server Architecture (2-Tier))
- client machines,
- These are the computers (PCs, laptops, mobile devices, etc.) that request services from a server.
- A client usually runs an application that connects to a database server or web server.
- They don't store the main database themselves—they just send requests and display results.
- E.g. A company employee's computer running a payroll application that fetches data from the central database server.

client machines & server machines

- server machines
- These are powerful computers dedicated to providing services (like storing data, running applications, handling requests).
- A server machine usually runs a Database Management System (DBMS) such as Oracle, MySQL, or SQL Server.
- It handles multiple clients at the same time.
- E.g. MySQL database server that stores all student records in a university.

three-tier architecture

- Three-tier architecture
  - is an extension of the client–server model.
  - It adds a **middle layer** between clients and the database.
- It has **3 layers**:
- **Presentation Layer (Client Machine)**
  - What the user interacts with (UI).
  - Example: Web browser, mobile app, or desktop GUI.
- **Application Layer (Middle Tier / Server)**
  - Business logic runs here.
  - Receives client requests, processes them, and sends DB queries to the database server.
  - Example: Web server running Java, Python (Django), or .NET code.
- **Data Layer (Database Server Machine)**
  - The actual DBMS where data is stored (Oracle, MySQL, SQL Server, etc.).

two-Tier and there-Tier architectures

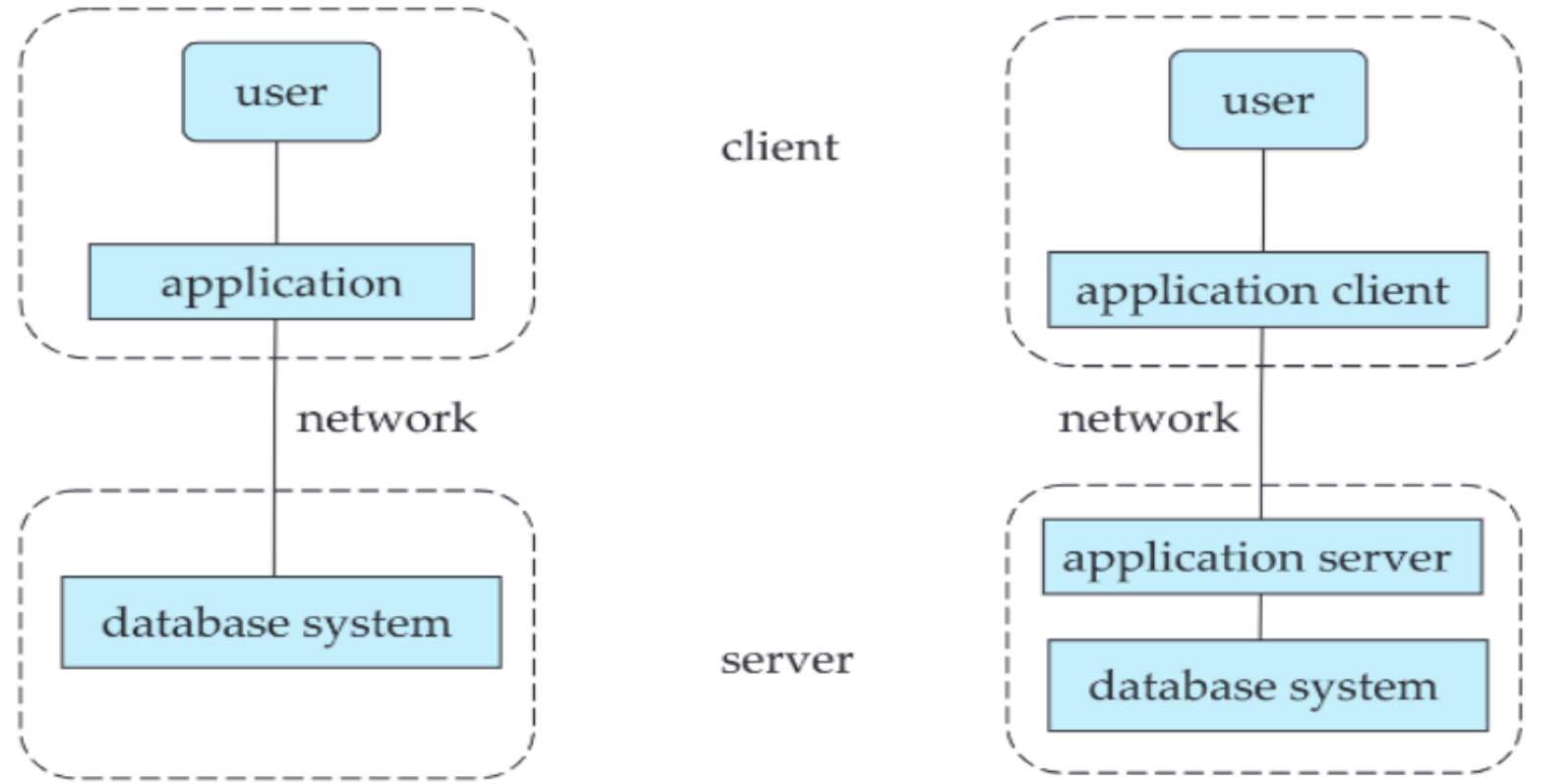


Fig. 1.3 : Two-Tier and Three-Tier Architectures


THANK
YOU


Any
Question?